МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №1

по дисциплине «Построение и анализ алгоритмов»

Тема: Поиск с возвратом

Вариант: 1и (Итеративный бэктрекинг)

Студент гр. 4343 _____ Едыгеев Р. Б.

Преподаватель _____ Калишенко Е. Л.

Санкт-Петербург

2026

Цель работы.

Необходимо реализовать итеративный алгоритм поиска с возвратом для задачи квадрирования квадрата.

Задание.

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до N−1, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу - квадрат размера N. Он может получить ее, собрав из уже имеющихся обрезков (квадратов).

Например, столешница размера 7×7 может быть построена из 9 обрезков.

Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные:

Размер столешницы - одно целое число N ($2 \le N \le 30$).

Выходные данные:

Одно число K, задающее минимальное количество обрезков (квадратов), из которых можно построить столешницу (квадрат) заданного размера N. Далее должны идти K строк, каждая из которых должна содержать три целых числа x, y и w, задающие координаты левого верхнего угла ($1 \le x,y \le N$) и длину стороны соответствующего обрезка (квадрата).

Основные теоретические положения.

Решение задачи методом поиска с возвратом сводится к последовательному расширению частичного решения. Если на очередном шаге такое расширение провести не удается, то возвращаются к более короткому частичному решению и продолжают поиск дальше. Данный

алгоритм позволяет найти все решения поставленной задачи, если они существуют. Для ускорения метода стараются вычисления организовать таким образом, чтобы как можно раньше выявлять заведомо неподходящие варианты. Зачастую это позволяет значительно уменьшить время нахождения решения.

Выполнение работы.

Для решения задачи методом итеративного бэктрекинга была разработана программа на языке C++, производящая поиск наилучшего решения (см. Приложение А).

Условно случаи задачи можно разбить на три: для четного N, для составного N, и для простого N. Отличие в том, что для четных N достаточно 4 квадратов, полностью и не пересекаясь заполняющих пространство. Для составных N достаточно узнать наименьший делитель, после чего найти решение для данного делителя и масштабировать данное решение. Однако для поиска решение при N простом единственный путь – перебор вариантов.

Для оптимизации было использовано несколько идей:

1. Битовые маски для быстрых операций с доской. Чтобы не хранить двумерный массив, можно использовать одномерный массив двоичных чисел, что значительно повышает производительность;
2. Префиксные маски. Используются для быстрых операций установки/удаления квадратов, а также для проверки возможности установки квадрата в заданную область;
3. Отсечение заведомо неподходящих решений. Если сумма текущей заполненности и отношения количества свободного пространства к площади покрытия максимального квадрата больше текущего лучшего решения – данный способ не рассматривается;
4. Итеративный поиск. Пошаговый поиск оптимального решения, который начинается с определенного, постепенно увеличивающегося числа квадратов, пока не будет найдено первое, то есть оптимальное решение.

Пайплайн работы алгоритма.

Программа начинается с чтения числа n. Сразу выполняется проверка на четность: если n четное, выводится тривиальное оптимальное решение из четырех квадратов размером n/2, расположенных в четырех углах исходного квадрата, после чего программа завершается.

Для нечетного n программа пытается найти наименьший делитель числа, начиная с двойки и перебирая до квадратного корня из n. Если находится делитель d, меньший самого числа, программа запоминает его как базовый размер для дальнейшего решения, иначе используется само число n.

Далее вычисляется коэффициент масштабирования как отношение исходного размера к найденному делителю. Затем создается доска размера d с использованием битового представления: каждая строка хранится как 32-битное целое число, где каждый бит соответствует одной клетке. Для оптимизации проверок инициализируется глобальный массив префиксных масок, позволяющий быстро получать битовую маску для любого диапазона колонок с помощью операции XOR.

После инициализации запускается процесс итеративного поиска решения для доски размера d. Последовательно увеличивается целевое количество квадратов, начиная с единицы. Для каждого целевого значения вызывается функция поиска, которая использует стек для обхода состояний в глубину.

Начальное решение: большой квадрат в углу и несколько маленьких по краям. Затем формируется стартовое состояние с тремя квадратами в углу и помещается в стек. Далее в цикле из стека извлекаются состояния и проверяются на заполненность доски, длину текущего решения относительно лучшего найденного и целевого количества, а также применяется отсечение по нижней границе на основе количества пустых клеток и максимально возможного размера квадрата в текущей позиции. Если состояние проходит все проверки, определяется первая пустая клетка, вычисляется максимально допустимый размер квадрата, и для каждого возможного размера от

максимального до единицы проверяется возможность размещения. При успешной проверке создается новое состояние с размещенным квадратом и помещается в стек для дальнейшего обхода.

Процесс продолжается до тех пор, пока либо не будет найдено решение с текущим целевым количеством квадратов, либо не будут исчерпаны все состояния. Когда решение найдено, оно сохраняется как лучшее, и цикл итеративного углубления переходит к следующей цели или завершается, если достигнуто решение.

После получения решения для размера d программа масштабирует его на исходный размер n путем умножения координат и размеров каждого квадрата на коэффициент масштабирования, корректируя координаты для перехода от нулевой индексации к единичной.

Анализ сложности алгоритма.

1. Для четных чисел сложность по времени, очевидно, равна O(1), так как просто выводятся 4 квадрата;

2. Для нечетных чисел включается алгоритм итеративного бэктрекинга, сложность по времени в общем случае – $O(b^d)$, где b – количество подходящих размеров квадратов в позиции, а d – количество размещенных квадратов. Иначе, временная сложность – $O(N^N) = O(2^{N\log N}) \approx O(2^N)$. При оптимизации, где начальный квадрат равен N/2 + 1, сложность будет $O(2^{N/2})$, ведь начинать алгоритм будет не с квадрата размером 1*1, а с квадрата (N/2 + 1)*(N/2 + 1), а также алгоритм будет отсекать заведомо проигрышные варианты, что существенно сокращает длительность поиска.

3. Пространственная сложность в лучшем случае равна O(1), т.к. ответ выводится сразу, алгоритм не запускается.

4. В общем случае необходимо хранить саму доску (O(N)) и стэк, память которого равна числу состояний, умноженному на размер одного состояния (d*N, т.е. O(d*N) для среднего, O(N^2) для худшего случая).

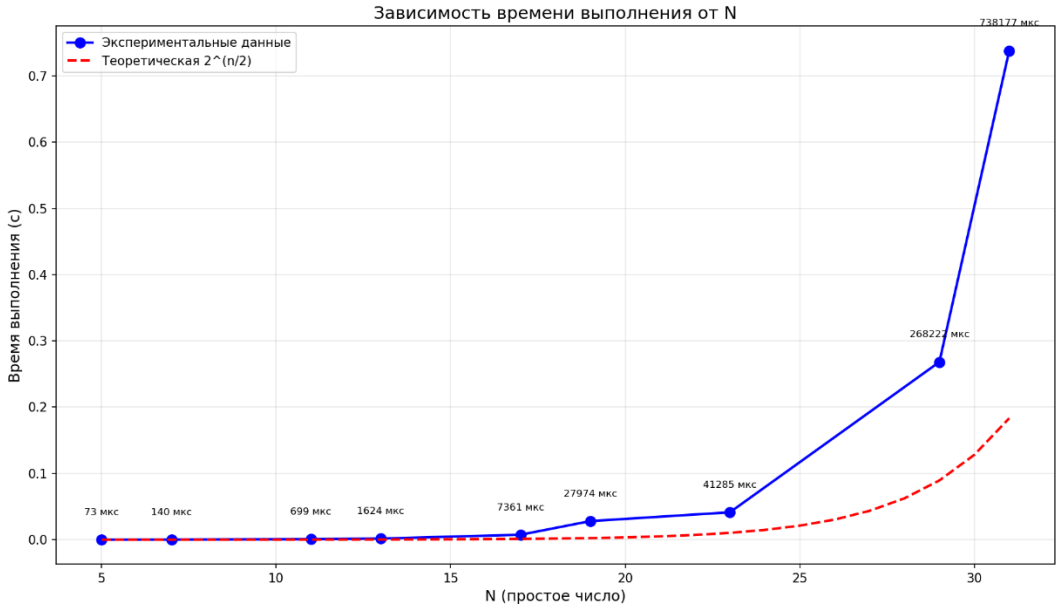

Рисунок 1 – Исследование сложности алгоритма

Как можно заметить, экспериментальные результаты недалеки от теоретических.

Тестирование.

Было проведено тестирование крайних случаев для разработанной программы.

1. Стандартный вывод (Рис. 2):

```
C:\Users\Hp\OneDrive\Рабочий стол\projects\2_COURSE\BAAA\lb1>g++ main.cpp -o main.exe && main.exe
19
13
1 1 10
1 11 9
11 1 9
10 11 1
10 12 1
10 13 2
10 15 5
11 10 2
11 12 1
12 12 3
13 10 2
15 10 5
15 15 5
```

Рисунок 2 – Штатный вывод программы

2. Ввод отрицательного значения, нуля, единицы (Рис. 3, 4, 5):

```
C:\Users\Hp\OneDrive\Рабочий стол\projects\2_COURSE\BAAA\lb1>g++ main.cpp -o main.exe && main.exe
-12
Неверное значение N
```

Рисунок 3 – Ввод отрицательного значения

```
C:\Users\Hp\OneDrive\Рабочий стол\projects\2_COURSE\BAAA\lb1>g++ main.cpp -o main.exe && main.exe
0
Неверное значение N
```

Рисунок 4 – Ввод нуля

```
C:\Users\Hp\OneDrive\Рабочий стол\projects\2_COURSE\BAAA\lb1>g++ main.cpp -o main.exe && main.exe
1
Неверное значение N
```

Рисунок 5 – Ввод единицы

Вывод.

В ходе выполнения лабораторной работы был изучен метод итеративного поиска с возвратом для решения задачи квадрирования квадрата. Была разработана программа на C++, выполнен анализ сложности алгоритма по времени и памяти, проведено тестирование программы. Был получен опыт построения алгоритма для решения задачи, а также опыт в сфере оценки и анализа проведенной работы.

ПРИЛОЖЕНИЕ А
КОД ПРОГРАММЫ

Файл main.cpp:

```
#include <iostream>
#include <vector>
#include <cmath>
#include <cstdint>
#include <stack>
#include <array>
#include <algorithm>

constexpr uint32_t createBitMask(int b) {
    return uint32_t(1) << b;
}

struct Square {
    int x, y, size;
};

static std::array<uint32_t, 33> BIT_PREFIXES;
static bool prefixes_initialized = false;

void initializePrefixes(int max_size) {
    BIT_PREFIXES[0] = 0;
    for (int i = 1; i <= max_size + 1; i++) {
        BIT_PREFIXES[i] = BIT_PREFIXES[i-1] | (1u << (i-1));
    }
    prefixes_initialized = true;
}

struct Board {
    int size;
    std::vector<uint32_t> cells;
    std::vector<Square> placed_squares;
    int minRow, minCol, maxRow, maxCol;

    Board(int n) : size(n), minRow(n), minCol(n), maxRow(-1), maxCol(-
1), cells(n, 0) {
        if (!prefixes_initialized) {
            initializePrefixes(n);
        }
    }

    bool isOccupied(int row, int col) const {
        return (cells[row] & (1u << col)) != 0;
    }

    bool firstEmpty(int &x, int &y) {
        uint32_t fullRowMask = (1u << size) - 1;
        for (int i = 0; i < size; i++) {
            if (cells[i] != fullRowMask) {
                for (int j = 0; j < size; j++) {
                    if (!(cells[i] & (1u << j))) {
                        x = i;
                        y = j;
                        return true;
```

```
                    }
                }
            }
        }
        return false;
    }

    bool canPlace(int x, int y, int squareSize) const {
        if (x + squareSize > size || y + squareSize > size) return
false;

        uint32_t mask = BIT_PREFIXES[y + squareSize] ^
BIT_PREFIXES[y];

        for (int i = x; i < x + squareSize; i++) {
            if (cells[i] & mask) return false;
        }
        return true;
    }

    void placeSquare(int x, int y, int squareSize) {
        uint32_t mask = BIT_PREFIXES[y + squareSize] ^
BIT_PREFIXES[y];

        for (int i = x; i < x + squareSize; i++) {
            cells[i] |= mask;
        }

        placed_squares.push_back({x, y, squareSize});
    }

    void removeSquare(int x, int y, int squareSize) {
        uint32_t mask = BIT_PREFIXES[y + squareSize] ^
BIT_PREFIXES[y];

        for (int i = x; i < x + squareSize; i++) {
            cells[i] &= ~mask;
        }
    }

    int emptyCellsCount() const {
        int count = 0;
        uint32_t fullRowMask = (1u << size) - 1;

        for (int i = 0; i < size; i++) {
            if (cells[i] == fullRowMask) continue;
            count += size - __builtin_popcount(cells[i]);
        }
        return count;
    }

    bool isRemainingSquare() {
        minRow = size + 1, minCol = size + 1, maxRow = -1, maxCol = -
1;
        for (int i = 0; i < size; ++i) {
            for (int j = 0; j < size; ++j) {
                if (!isOccupied(i, j)) {
```

```
                    minRow = std::min(minRow, i);
                    minCol = std::min(minCol, j);
                    maxRow = std::max(maxRow, i);
                    maxCol = std::max(maxCol, j);
                }
            }
        }

        if (maxRow == -1) return false;

        int side = maxRow - minRow + 1;
        if (side != maxCol - minCol + 1) return false;

        return canPlace(minRow, minCol, side);
    }
};

int getMaxSquareSize(int n) {
    return (n/2) + 1;
}

void findSolution(Board initialBoard, int& minSquares, int
targetCount, std::vector<Square>& bestSolution) {
    std::stack<Board> states;
    std::vector<Square> currentSolution;

    std::vector<Square> heuristic;
    heuristic.push_back({0, 0, initialBoard.size - 1});
    for (int i = 0; i < initialBoard.size; i++)
heuristic.push_back({initialBoard.size - 1, i, 1});
    for (int i = 0; i < initialBoard.size - 1; i++)
heuristic.push_back({i, initialBoard.size - 1, 1});

    if (heuristic.size() < bestSolution.size() ||
bestSolution.empty()) {
        bestSolution = heuristic;
        minSquares = heuristic.size();
    }

    int bestSide = getMaxSquareSize(initialBoard.size);
    Board tempBoard = initialBoard;
    tempBoard.placeSquare(0, 0, bestSide);
    tempBoard.placeSquare(0, bestSide, initialBoard.size - bestSide);
    tempBoard.placeSquare(bestSide, 0, initialBoard.size - bestSide);
    states.push(tempBoard);

    while (!states.empty()) {
        Board current = states.top();
        states.pop();

        int x, y;
        if (!current.firstEmpty(x, y)) {
            if (current.placed_squares.size() < bestSolution.size()) {
                bestSolution = current.placed_squares;
                minSquares = bestSolution.size();
            }
            continue;
```

```
        }

        if (current.placed_squares.size() >= bestSolution.size() ||
            current.placed_squares.size() > (size_t)targetCount)
continue;

        int max_size = std::min({initialBoard.size - x,
initialBoard.size - y});

        if (current.placed_squares.size() + current.emptyCellsCount()
/ (max_size * max_size) >= bestSolution.size())
            continue;

        while (max_size > 0 && !current.canPlace(x, y, max_size))
max_size--;

        for(int size = max_size; size >= 1; size--) {
            if (current.canPlace(x, y, size)) {
                current.placeSquare(x, y, size);
                states.push(current);
                current.placed_squares.pop_back();
                current.removeSquare(x, y, size);
            }
        }
    }
}

void solve(int n, std::vector<Square>& bestResult) {
    Board board(n);
    int minSquares = n * n + 1;

    for (int targetCount = 1; targetCount <= minSquares;
++targetCount) {
        findSolution(board, minSquares, targetCount, bestResult);
        if (minSquares <= targetCount) break;
    }
}

int main() {
    int n;
    if (!(std::cin >> n)) return 0;
    if (n <= 1) {
        std::cout << "Неверное значение N" << std::endl;
        return 0;
    }

    if (n % 2 == 0) {
        int half = n / 2;
        std::vector<Square> solution = {
            {1, 1, half},
            {1, half + 1, half},
            {half + 1, 1, half},
            {half + 1, half + 1, half}
        };
        std::cout << solution.size() << "\n";
        for (const auto& square : solution) {
```

```
            std::cout << square.x << " " << square.y << " " <<
square.size << "\n";
        }
        return 0;
    }

    int divisor = n;
    for (int d = 2; d * d <= n; ++d) {
        if (n % d == 0) {
            divisor = d;
            break;
        }
    }

    int scale = n / divisor;

    std::vector<Square> smallResult;
    solve(divisor, smallResult);

    std::vector<Square> finalResult;
    finalResult.reserve(smallResult.size());
    for (const auto& square : smallResult) {
        finalResult.push_back({square.x * scale + 1, square.y * scale
+ 1, square.size * scale});
    }

    std::cout << finalResult.size() << "\n";
    for (const auto& square : finalResult) {
        std::cout << square.x << " " << square.y << " " << square.size
<< "\n";
    }

    return 0;
}
```